МИНОБРНАУКИ РОССИИ

Санкт-Петербургский Государственный Электротехнический Университет

«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра Информационные системы

ОТЧЕТ

по лабораторной работе № 1

по дисциплине: «Информатика и информационные технологии»

Тема: «Простой интерпретатор арифметических действий»

Студент гр. 6373 : _____ / Сагутдинов А.В.

Преподаватель: _____ / Дюков Н.В.

Санкт-Петербург

2026

Цель работы

Написать код на языке Python, представляющий собой интерпретатор арифметических выражений.

Основные теоретические положения

1. Нотации записи математических выражений

В вычислительной математике и теории трансляции языков программирования выделяют три ключевые формы представления математических выражений:

1. **Инфиксная нотация:** общепринятая форма записи, в которой бинарный оператор располагается между своими операндами (A + B). Главным недостатком инфиксной записи для автоматической обработки ЭВМ является неоднозначность порядка выполнения операций, требующая явного задания правил приоритетов, ассоциативности и использования круглых скобок для принудительного переопределения порядка действий.

2. **Постфиксная нотация (обратная польская запись, ОПЗ, RPN):** форма записи, предложенная австралийским учёным Ч. Хэмблином на основе польской нотации Я. Лукасевича, при которой оператор располагается строго после операндов (A B +).

3. **Префиксная нотация (прямая польская нотация Лукасевича):** форма записи, в которой оператор располагается перед операндами (+ A B).

Преимущества префиксной и постфиксной форм:

- Полное отсутствие необходимости в скобках;

- Порядок применения операторов однозначно определяется их положением в строке;

- Вычисление выражения на базе стека выполняется за один последовательный проход с линейной вычислительной сложностью O(n), где n — количество токенов.

2. Абстрактный тип данных «стек»

Стек (Stack) — динамическая структура данных, функционирующая по дисциплине LIFO (Last In, First Out — «последним пришёл, первым ушёл»). Доступ осуществляется исключительно к вершине стека.

Базовые операции:

- push — добавление элемента на вершину стека;

- pop — извлечение и удаление элемента с вершины стека;

- peek (или обращение по индексу [-1]) — чтение элемента на вершине без его удаления.

В классическом интерпретаторе выражений используются два независимых стека:

1. Стек операндов (nums): накапливает числовые значения (константы и промежуточные результаты вычислений);

2. Стек операторов (opers): хранит символы операций, функции и открывающие скобки, ожидающие вычисления в соответствии со своими приоритетами.

3. Приоритеты и ассоциативность операторов

Приоритет определяет последовательность выполнения операций при отсутствии скобок. Ассоциативность определяет порядок применения операций с одинаковым приоритетом:

- Левоассоциативные операции выполняются слева направо: a - b - c = (a - b) - c.

- Правоассоциативные операции выполняются справа налево: -(-a) = -(-a).

Таблица приоритетов

Оператор	Назначение	Приоритет	Ассоциативность
==	Проверка на равенство	0	Левая
+, -	Бинарное сложение и вычитание	1	Левая
*, /	Бинарное умножение и деление	2	Левая

Оператор	Назначение	Приоритет	Ассоциативность
u-	Унарный минус (инверсия знака)	3	Правая
cos	Тригонометрическая функция косинуса	Высший (функция)	Применяется к аргументу в скобках

4. Особенности лексического анализа и синтаксического разбора

1. Токенизация многозначных и вещественных чисел: символы цифр 0–9 и десятичной точки '.' аккумулируются в буферную переменную cur_num до появления очередного оператора, после чего преобразуются в тип float. Это предотвращает разбиение составных чисел (например, 24.5) на отдельные цифры.

2. Идентификация унарного минуса (u-): так как символ дефиса '-' используется как для бинарного вычитания, так и для изменения знака числа, применяется контекстный анализ:

- Минус является унарным (u-), если он расположен в самом начале строки (индекс i = 0), либо непосредственно следует за открывающей скобкой '(' или любым оператором (+, -, *, /, ==).

- Если перед минусом стоит закрывающая скобка ')' или число, он признаётся бинарным вычитанием.

3. Обработка составных операторов и функций:

- Двухсимвольный оператор сравнения == распознаётся методом заглядывания вперёд: при обнаружении первого символа '=' проверяется равенство следующего символа s[i + 1] == '=', после чего они склеиваются в единый токен, а второй символ пропускается.

- Буквенные идентификаторы функций накапливаются в строковом буфере cur_word до распознавания ключевого слова (cos), после чего помещаются в стек операторов.

Разработка программы

Программа разработана на языке Python и состоит из следующих ключевых функциональных модулей:

1. **Функция calculate(nums, operator):** реализует выполнение математических операций над вершиной стека операндов. Для унарных операций (u-, cos) извлекается ровно один операнд (nums.append(-nums.pop()) или nums.append(cos(nums.pop()))). Для бинарных операций последовательно извлекаются два операнда (второй и первый соответственно) и результат операции заносится в стек.

2. **Функция to_postfix(s):** реализует трансляцию инфиксного выражения в постфиксную форму записи (ОПЗ). Многозначные и вещественные числа накапливаются в строковом буфере cur_num и выводятся как отдельные токены, разделяемые пробелами.

3. **Функция to_prefix(s):** реализует преобразование постфиксного потока токенов в префиксную форму выражения с использованием стека подвыражений. Для каждой операции формируется структура [оператор] [операнд_1] [операнд_2].

4. **Функция parser(s):** основной интерпретатор, реализующий синтаксический анализ и параллельное вычисление значения выражения на двух стеках в один проход.

Результаты работы программы

Для подтверждения корректности реализованных алгоритмов выполнено комплексное тестирование интерпретатора на наборе выражений, покрывающих все граничные условия.

№	Входное выражение	Постфиксная запись (ОПЗ)	Префиксная запись	Результат	Проверяемое свойство
1	2 + 3 * 4	2 3 4 * +	+ 2 * 3 4	14.0	Базовый приоритет умножения над сложением
2	(2 + 3) * 4	2 3 + 4 *	* + 2 3 4	20.0	Изменение порядка вычислений круглыми скобками

№	Входное выражение	Постфиксная запись (ОПЗ)	Префиксная запись	Результат	Проверяемое свойство
3	(2 + 3) * (4 - 1)	2 3 + 4 1 - *	* + 2 3 - 4 1	15.0	Вложенные скобки
4	10 - 4 - 2	10 4 - 2 -	- - 10 4 2	4.0	Левая ассоциативность вычитания ((10 - 4) - 2)
5	7.5 / 2.5 + 1.2	7.5 2.5 / 1.2 +	+ / 7.5 2.5 1.2	4.2	Корректная обработка дробных чисел (float)
6	-5 + 3	5 u- 3 +	+ u- 5 3	-2.0	Унарный минус в начале выражения
7	3 * -5	3 5 u- *	* 3 u- 5	-15.0	Унарный минус после

№	Входное выражение	Постфиксная запись (ОПЗ)	Префиксная запись	Результат	Проверяемое свойство
					оператора умножения
8	-(2 + 3) * 4	2 3 + u- 4 *	* u- + 2 3 4	-20.0	Унарный минус перед открывающей скобкой
9	(2 + 2) - 1	2 2 + 1 -	- + 2 2 1	3.0	Бинарный минус после закрывающей скобки
10	cos(0) + 5	0 cos 5 +	+ cos 0 5	6.0	Вычисление функции cos(x) по Варианту 19
11	2 + 2 == 4	2 2 + 4 ==	== + 2 2 4	True (1.0)	Оператор равенства == (истинное утверждение

№	Входное выражение	Постфиксная запись (ОПЗ)	Префиксная запись	Результат	Проверяемое свойство
					)
12	2 + 2 == 5	2 2 + 5 ==	== + 2 2 5	False (0.0)	Оператор равенства == (ложное утверждение)
13	cos(0) == 1	0 cos 1 ==	== cos 0 1	True (1.0)	Совместная обработка функции cos и оператора ==

Пример протокола работы программы в терминале:

```
Введите выражение: (2 + 3) * (4 - 1)
Результат:         15.0
Постфиксная запись: 2 3 + 4 1 - *
Префиксная запись:  * + 2 3 - 4 1

Введите выражение: cos(0) == 1
Результат:         True
Постфиксная запись: 0 cos 1 ==
Префиксная запись:  == cos 0 1
```

Выводы

В ходе выполнения лабораторной работы № 1 были детально исследованы и реализованы принципы стекового анализа и вычисления арифметических выражений. Созданный программный модуль обеспечивает корректный парсинг инфиксных выражений, поддерживает приоритеты базовых бинарных операций, произвольную вложенность круглых скобок, отрицательные и вещественные числа.

В рамках индивидуального задания успешно интегрированы вычисление математической функции cos(x) и операция сравнения ==. В качестве усложнения разработаны алгоритмы двусторонней конвертации в постфиксную (ОПЗ) и префиксную нотации. Все тестовые сценарии выполнены с получением корректных результатов, полностью соответствующих теоретическим расчётам.

ПРИЛОЖЕНИЕ

Исходный код программы (lab1.py):

`https://github.com/corequadz/study/blob/main/labs/%D0%98%D0%BD%D1%84%D0%98%D0%A2/lab1.py`